

EVIDENCIA DE DESARROLLO: APP ESTACIONAMIENTO

POR BLANCA MICHELLE RODRIGUEZ CHAVARRIA

Descripción

Esta aplicación fue desarrollada con Spring Boot, Maven y Hibernate (JPA) bajo una arquitectura por capas (dominio, DAO, servicio y controlador). Permite gestionar entradas y salidas de vehículos en un estacionamiento, generando reportes mensuales para residentes y reseteando información automáticamente. Utiliza APIs REST, facilitando su integración con futuras interfaces de usuario o sistemas externos.

1. Base de datos

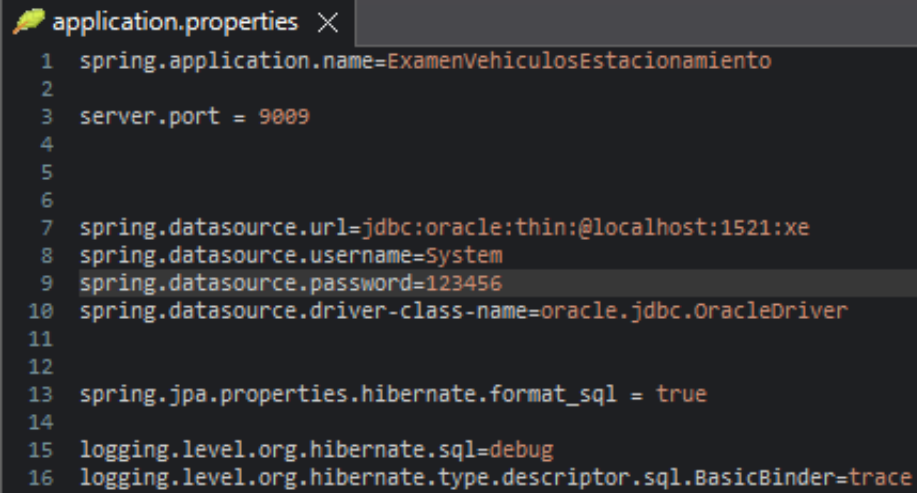
Se utilizó una base de datos en Oracle con las siguientes entidades:

- **Vehículo:** placa, tipo de vehículo, minutos acumulados.
- **Estancia:** fecha de entrada, fecha de salida, relación con vehículo.

```
CREATE TABLE VEHICULO_EXAMEN (  
  placa VARCHAR(20) PRIMARY KEY,  
  tipo VARCHAR2(20) CHECK (tipo IN ('OFICIAL', 'RESIDENTE', 'NO_RESIDENTE')),  
  entrada_actual DATE DEFAULT NULL,  
  minutos_acumulados INT DEFAULT 0  
);
```

```
CREATE TABLE ESTANCIA_EXAMEN (  
  id NUMBER PRIMARY KEY,  
  entrada DATE NOT NULL,  
  salida DATE NOT NULL,  
  vehiculo_placa VARCHAR(20),  
  CONSTRAINT fk_vehiculo FOREIGN KEY (vehiculo_placa)  
    REFERENCES VEHICULO_EXAMEN(placa)  
);
```

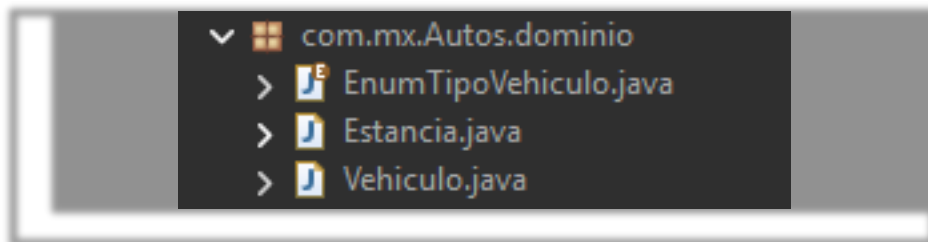
Se configuró application.properties con los parámetros para JPA

A screenshot of a code editor window titled 'application.properties' with a close button. The window contains 16 lines of configuration code for a Spring application. The code is as follows:

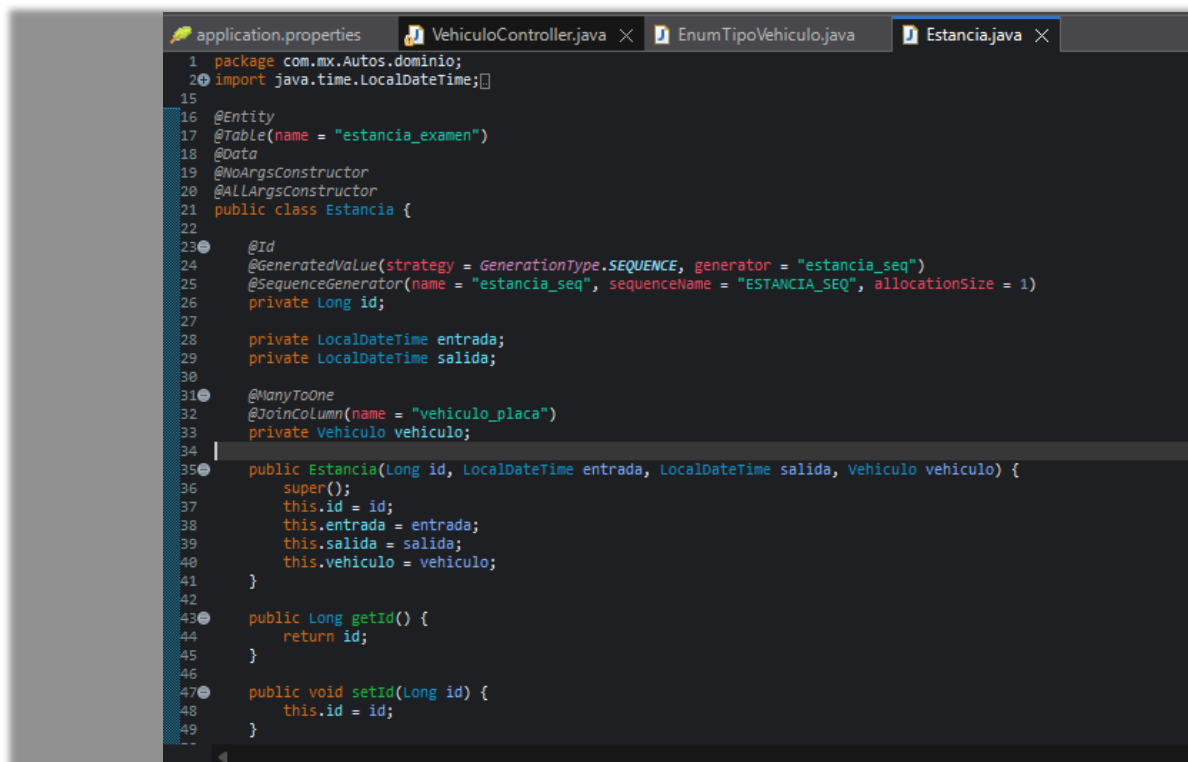
```
1 spring.application.name=ExamenVehiculosEstacionamiento
2
3 server.port = 9009
4
5
6
7 spring.datasource.url=jdbc:oracle:thin:@localhost:1521:xe
8 spring.datasource.username=System
9 spring.datasource.password=123456
10 spring.datasource.driver-class-name=oracle.jdbc.OracleDriver
11
12
13 spring.jpa.properties.hibernate.format_sql = true
14
15 logging.level.org.hibernate.sql=debug
16 logging.level.org.hibernate.type.descriptor.sql.BasicBinder=trace
```

2. Dominio

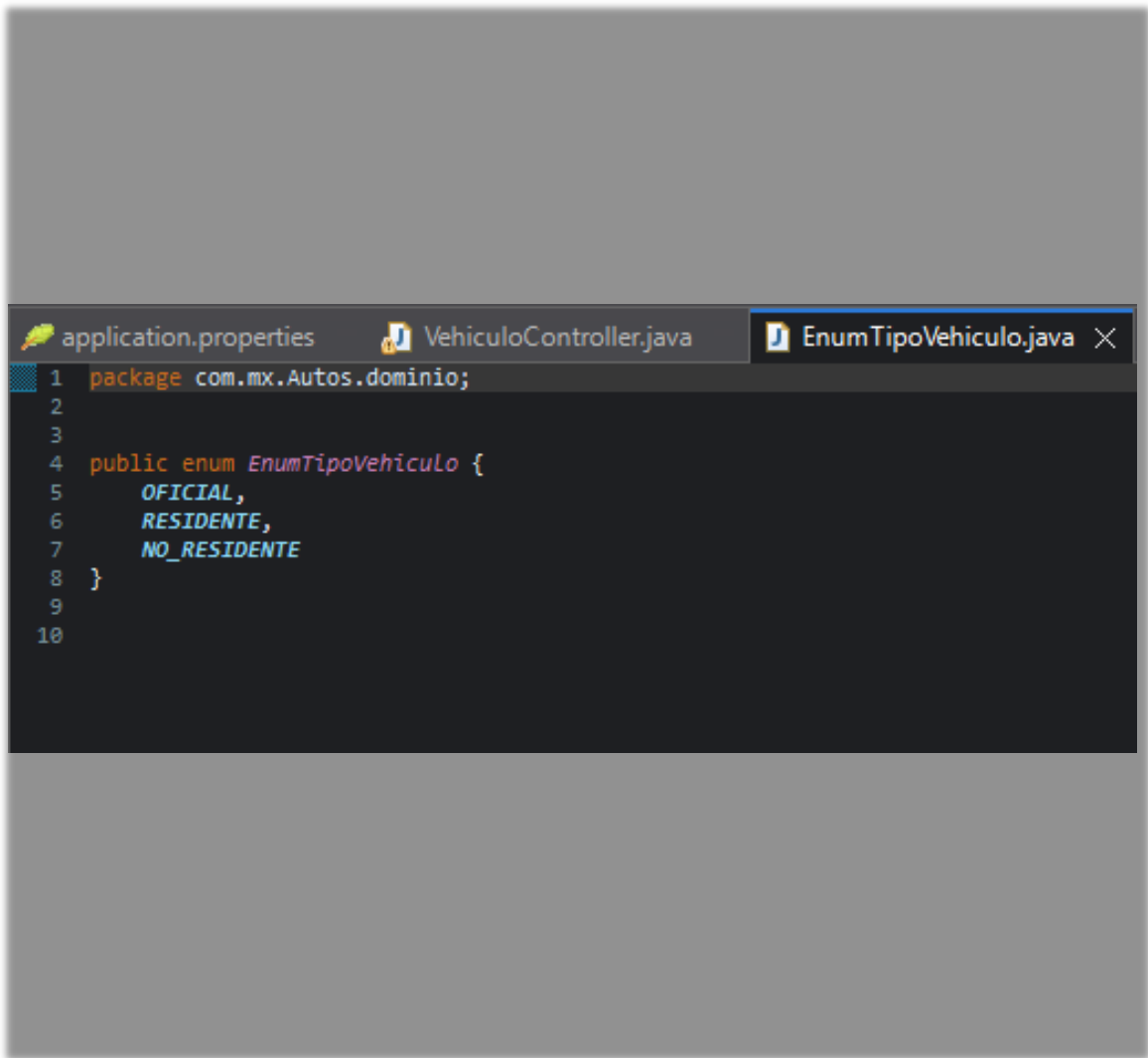
Dentro del paquete `com.mx.estacionamiento.dominio` se definieron las entidades: `Vehiculo.java`, `Estancia.java` y `Enumtipovehiculo`



Ambas están anotadas con `@Entity` y contienen las relaciones necesarias, getters/setters, y anotaciones de JPA



```
application.properties  VehiculoController.java  EnumTipoVehiculo.java  Estancia.java
1 package com.mx.Autos.dominio;
2
3 import java.time.LocalDateTime;
16
17 @Entity
18 @Table(name = "VEHICULO_EXAMEN")
19 @Data
20 @NoArgsConstructor
21 @AllArgsConstructor
22 public class Vehiculo {
23
24     @Id
25     private String placa;
26
27     @Column(nullable = false)
28     private String tipo;
29
30     @Column(name = "entrada_actual")
31     private LocalDateTime entradaActual;
32
33     @Column(name = "minutos_acumulados")
34     private int minutosAcumulados;
35
36     @OneToMany(mappedBy = "vehiculo", cascade = CascadeType.ALL, orphanRemoval = true)
37     private List<Estancia> estancias = new ArrayList<>();
38
39     public String getPlaca() {
40         return placa;
41     }
42
43     public void setPlaca(String placa) {
44         this.placa = placa;
45     }
46
47     public String getTipo() {
48         return tipo;
49     }
50 }
```

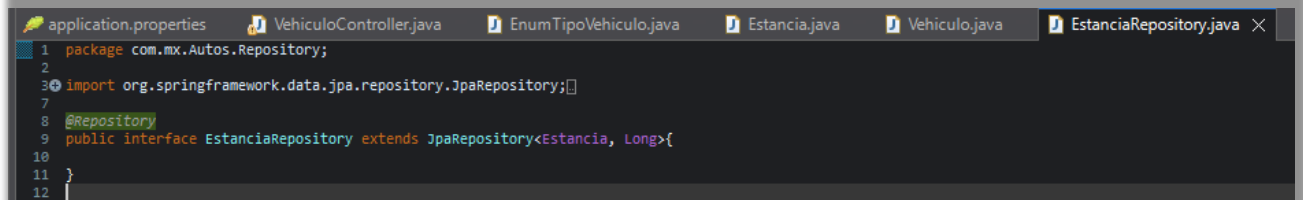


```
application.properties  VehiculoController.java  EnumTipoVehiculo.java X
1 package com.mx.Autos.dominio;
2
3
4 public enum EnumTipoVehiculo {
5     OFICIAL,
6     RESIDENTE,
7     NO_RESIDENTE
8 }
9
10
```

3. DAO Repository

En el paquete com.mx.estacionamiento.dao se declararon los siguientes repositorios:

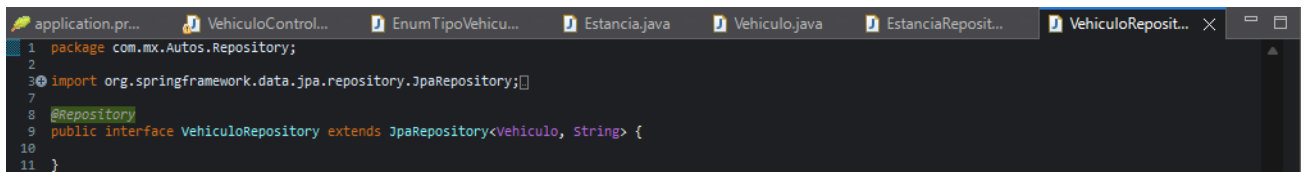
VehiculoRepository y EstanciasRepository ambos extienden JpaRepository y contienen personalizados como:



```

1 package com.mx.Autos.Repository;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5
6
7
8 @Repository
9 public interface EstanciaRepository extends JpaRepository<Estancia, Long>{
10
11 }
12

```



```

1 package com.mx.Autos.Repository;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5
6
7
8 @Repository
9 public interface VehiculoRepository extends JpaRepository<Vehiculo, String> {
10
11 }

```

4 .Service

En com.mx.estacionamiento.service se creó la clase Implementacion.java donde se implemento los caso de uso

- VehiculoService

```
application.properties  EstanciaRepository.java  VehiculoService.java X
1 package com.mx.Autos.servicio;
2
3 import java.time.Duration;
4
5
6
7
8
9
10
11
12
13
14
15
16
17 @Service
18 @RequiredArgsConstructor
19 public class VehiculoService {
20
21     @Autowired
22     VehiculoRepository vehiculoRepo;
23     @Autowired
24     EstanciaRepository estanciaRepo;
25
26     public void registrarEntrada(String placa) {
27         Vehiculo v = vehiculoRepo.findById(placa).orElseThrow();
28         v.setEntradaActual(LocalDateTime.now());
29         vehiculoRepo.save(v);
30     }
31
32     public double registrarSalida(String placa) {
33         Vehiculo v = vehiculoRepo.findById(placa).orElseThrow();
34         LocalDateTime salida = LocalDateTime.now();
35         long minutos = Duration.between(v.getEntradaActual(), salida).toMinutes();
36
37         if ("OFICIAL".equals(v.getTipo())) {
38             estanciaRepo.save(new Estancia(null, v.getEntradaActual(), salida, v));
39         } else if ("RESIDENTE".equals(v.getTipo())) {
40             v.setMinutosAcumulados(v.getMinutosAcumulados() + (int) minutos);
41         } else if ("NO_RESIDENTE".equals(v.getTipo())) {
42             return minutos * 0.5;
43         }
44
45         v.setEntradaActual(null);
46         vehiculoRepo.save(v);
47         return 0;
48     }
49
50 }
```

1. Controller

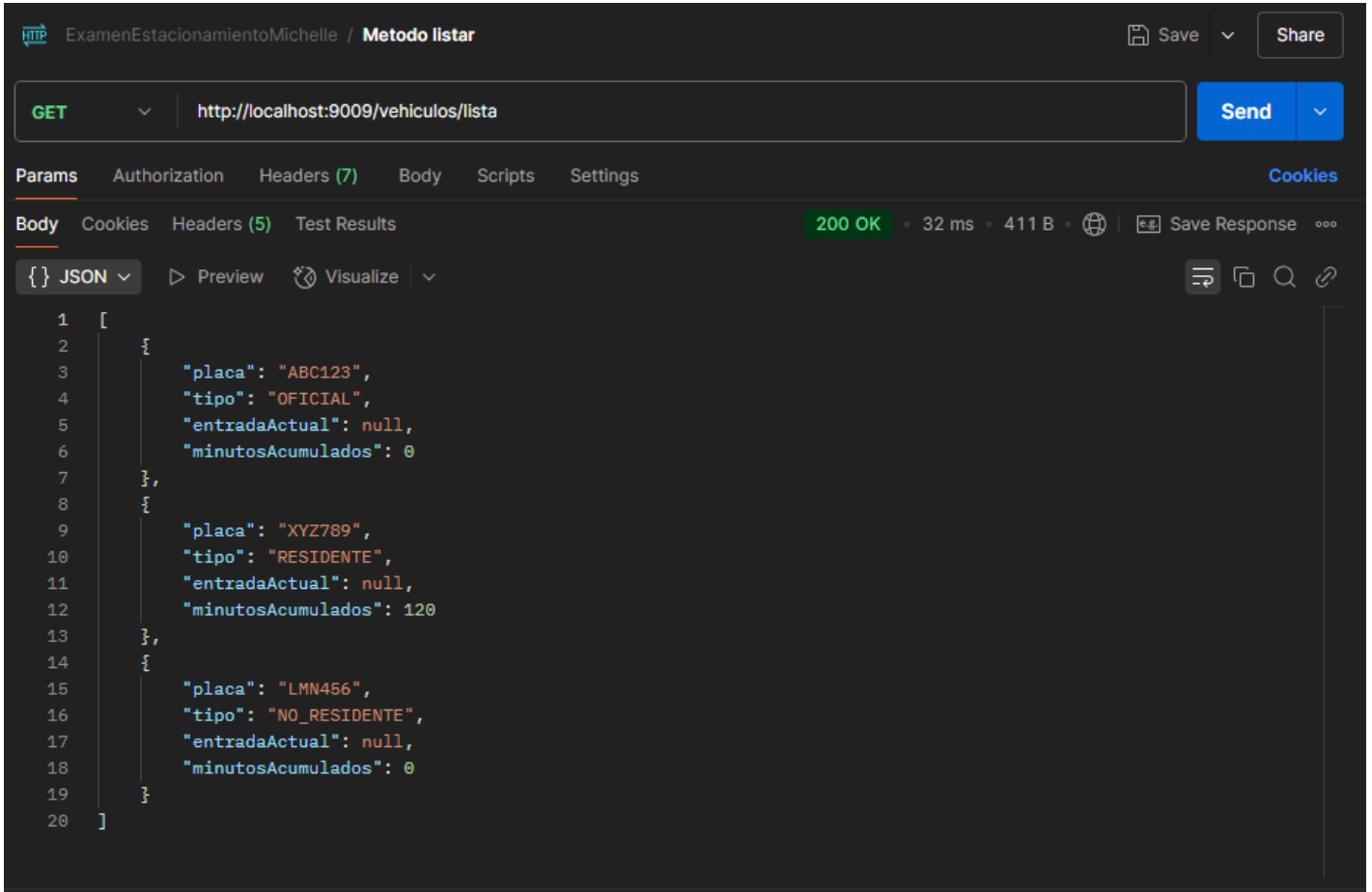
En el paquete controller se creó la clase VehiculoController



```
1 package com.mx.Autos.controller;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4
5 @RestController
6 @RequestMapping("/vehiculos")
7 @RequiredArgsConstructor
8 public class VehiculoController {
9
10     @Autowired
11     VehiculoService service;
12
13     // URL: http://localhost:9001/vehiculos/lista
14     @GetMapping(value = "/lista")
15     public ResponseEntity<?> lista() {
16         try {
17             return ResponseEntity.status(HttpStatus.OK).body(service.listarVehiculos());
18         } catch (Exception e) {
19             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
20                 .body("Error al obtener los vehiculos: " + e.getMessage());
21         }
22     }
23
24     // URL: http://localhost:9001/vehiculos/entrada/
25     @PostMapping(value = "entrada/{placa}")
26     public ResponseEntity<String> entrada(@PathVariable("placa") String placa) {
27         try {
28             service.registrarEntrada(placa);
29             return ResponseEntity.ok("Entrada registrada");
30         } catch (Exception e) {
31             return ResponseEntity.status(HttpStatus.BAD_REQUEST)
32                 .body("Error al registrar la entrada: " + e.getMessage());
33         }
34     }
35
36     // URL: http://localhost:9001/vehiculos/salida/
37     @PostMapping(value = "salida/{placa}")
38     public ResponseEntity<String> salida(@PathVariable("placa") String placa) {
39         try {
40             double pago = service.registrarSalida(placa);
41             return ResponseEntity.ok("Salida registrada. Pago: $" + pago);
42         } catch (Exception e) {
43             return ResponseEntity.status(HttpStatus.BAD_REQUEST)
44                 .body("Error al registrar la salida: " + e.getMessage());
45         }
46     }
47 }
```


2. Pruebas de funcionalidad

Las pruebas fueron realizadas en Postman se probaron los siguientes escenarios:



The screenshot shows the Postman interface for a GET request to `http://localhost:9009/vehiculos/lista`. The response status is **200 OK** with a response time of 32 ms and a body size of 411 B. The response body is displayed in JSON format, showing an array of three vehicle objects.

```
1  [
2    {
3      "placa": "ABC123",
4      "tipo": "OFICIAL",
5      "entradaActual": null,
6      "minutosAcumulados": 0
7    },
8    {
9      "placa": "XYZ789",
10     "tipo": "RESIDENTE",
11     "entradaActual": null,
12     "minutosAcumulados": 120
13   },
14   {
15     "placa": "LMN456",
16     "tipo": "NO_RESIDENTE",
17     "entradaActual": null,
18     "minutosAcumulados": 0
19   }
20 ]
```

